

HDOC DAY – 20

Question

Write a menu-driven modular C program to calculate geometrical measurements and physical properties of different solids using functions, decision-making statements, and loops. The program should support the following solids: Sphere, Cylinder, Cone, and Cuboid. It should provide these menu operations:

1. Calculate Surface Area
2. Calculate Volume
3. Calculate Base Area
4. Calculate Mass using density and volume
5. Calculate Weight
6. Calculate Gravitational Potential Energy
7. Calculate Pressure on the base
8. Calculate Buoyant Force
9. Determine Buoyancy Behaviour — Floats, Suspended, or Sinks

The project must be divided into separate modules: `main.c`, `geometry.c`, `physics.c`, `math_utils.c`, and `solid.h`. Use $\pi = 3.1416$, $\text{GRAVITY} = 9.8$, and $\text{TOLERANCE} = 0.000001$. Do not use functions from `math.h`; calculate square roots using the Newton-Raphson method. All inputs must be validated, and the program should continue displaying the menu until the user selects Exit.

AIM:

To develop a menu-driven modular C program using functions, selection statements, and loops to calculate geometrical measurements and physical properties of different solids such as sphere, cylinder, cone, and cuboid.

The program will calculate surface area, volume, base area, mass, weight, gravitational potential energy, pressure, buoyant force, and buoyancy behaviour.

The project must **not** use arrays, pointers, structures, recursion, or functions from `math.h`.

>ALGORITHM:

Algorithm 1 — Surface Area of Sphere

Function: `sphere_area(double radius)`

1. Start.

2. Input the radius of the sphere.
3. Check that radius is greater than 0.
4. Calculate:

Surface Area = $4 \times \text{PI} \times \text{radius} \times \text{radius}$

5. Return the surface area.
6. Stop.

Algorithm 2 — Volume of Sphere

Function: sphere_volume(double radius)

1. Start.
2. Receive the radius of the sphere.
3. Calculate:

Volume = $(4.0 / 3.0) \times \text{PI} \times \text{radius} \times \text{radius} \times \text{radius}$

4. Return the calculated volume.
5. Stop.

Algorithm 3 — Surface Area of Cylinder

Function: cylinder_area(double radius, double height)

1. Start.
2. Receive the radius and height of the cylinder.
3. Calculate:

Surface Area = $2 \times \text{PI} \times \text{radius} \times (\text{radius} + \text{height})$

4. Return the calculated surface area.
5. Stop.

Algorithm 4 — Volume of Cylinder

Function: cylinder_volume(double radius, double height)

1. Start.
2. Receive the radius and height of the cylinder.

3. Calculate:

Volume = $\text{PI} \times \text{radius} \times \text{radius} \times \text{height}$

4. Return the calculated volume.
5. Stop.

Algorithm 5 — Surface Area of Cone

Function: cone_area(double radius, double height)

1. Start.
2. Receive the radius and height of the cone.
3. Calculate:

number = $\text{radius} \times \text{radius} + \text{height} \times \text{height}$

4. Call calculate_square_root(number) to calculate the **slant height**.
5. Calculate:

Surface Area = $\text{PI} \times \text{radius} \times (\text{radius} + \text{slant height})$

6. Return the calculated surface area.
7. Stop.

Algorithm 6 — Volume of Cone

Function: cone_volume(double radius, double height)

1. Start.
2. Receive the radius and height of the cone.
3. Calculate:

Volume = $(1.0 / 3.0) \times \text{PI} \times \text{radius} \times \text{radius} \times \text{height}$

4. Return the calculated volume.
5. Stop.

Algorithm 7 — Surface Area of Cuboid

Function: cuboid_area(double length, double breadth, double height)

1. Start.
2. Receive the **length, breadth, and height** of the cuboid.
3. Calculate:

Surface Area = $2 \times (\text{length} \times \text{breadth} + \text{breadth} \times \text{height} + \text{length} \times \text{height})$

4. Return the calculated surface area.
5. Stop.

Algorithm 8 — Volume of Cuboid

Function: cuboid_volume(double length, double breadth, double height)

1. Start.
2. Receive the **length, breadth, and height** of the cuboid.
3. Calculate:

Volume = $\text{length} \times \text{breadth} \times \text{height}$

4. Return the calculated volume.
5. Stop.

Algorithm 9 — Base Area of Cylinder and Cone

Function: circular_base_area(double radius)

1. Start.
2. Receive the **radius**.
3. Calculate:

Base Area = $\text{PI} \times \text{radius} \times \text{radius}$

4. Return the calculated base area.
5. Stop.

Algorithm 10 — Base Area of Cuboid

Function: cuboid_base_area(double length, double breadth)

1. Start.
2. Receive the **length** and **breadth** of the cuboid.
3. Calculate:

Base Area = length × breadth

4. Return the calculated base area.
5. Stop.

Algorithm 11 — Calculate Mass of a Solid

Function: calculate_mass(double density)

1. Start.
2. Receive the **density** of the solid.
3. Get the **type of solid**:
 - Sphere
 - Cylinder
 - Cone
 - Cuboid
4. Obtain the required dimensions of the selected solid.
5. Calculate the volume by calling the appropriate volume function.
6. Calculate:

Mass = Density × Volume

7. Return the calculated mass in **kilograms (kg)**.
8. Stop.

Algorithm 12 — Calculate Weight of a Solid

Function: calculate_weight()

1. Start.
2. Call calculate_mass() to obtain the **mass of the solid**.
3. Calculate:

$$\text{Weight} = \text{Mass} \times \text{GRAVITY}$$

where **GRAVITY** = **9.8 m/s²**.

4. Return the calculated weight in **newtons (N)**.
5. Stop.

Algorithm 13 — Gravitational Potential Energy

Function: potential_energy(double elevation)

1. Start.
2. Receive the **elevation**.
3. Call calculate_mass() to obtain the **mass of the solid**.
4. Calculate:

Potential Energy = Mass × GRAVITY × Elevation

where **GRAVITY = 9.8 m/s²**.

5. Return the calculated potential energy in **joules (J)**.
6. Stop.

Algorithm 14 — Calculate Pressure on the Base

Function: calculate_pressure(double density)

1. Start.
2. Receive the **density** of the solid.
3. Get the type of solid:
 - Cylinder
 - Cone
 - Cuboid
4. Obtain the required dimensions.
5. Calculate the **contact area**:
 - For cylinder or cone, call circular_base_area().
 - For cuboid, call cuboid_base_area().
6. Calculate the **volume** by calling the appropriate volume function.
7. Calculate:

Pressure = (Density × Volume × GRAVITY) / Contact Area

8. Return the calculated pressure in **pascals (Pa)**.
9. Stop.

Algorithm 15 — Calculate Buoyant Force

Function: buoyant_force(double fluid_density)

1. Start.
2. Receive the **fluid density**.
3. Get the type of solid:
 - Sphere
 - Cylinder
 - Cone
 - Cuboid
4. Obtain the required dimensions of the selected solid.
5. Calculate the **volume** by calling the appropriate volume function.
6. Calculate:

Buoyant Force = Fluid Density × Volume × GRAVITY

7. Return the calculated buoyant force in **newtons (N)**.
8. Stop.

Algorithm 16 — Determine Buoyancy Behaviour

Function: determine_buoyancy(double solid_density, double fluid_density)

1. Start.
2. Receive the **solid density** and **fluid density**.
3. Compare both densities.
4. If **solid density > fluid density**, return **SINKS**.
5. If **solid density < fluid density**, return **FLOATS**.
6. If **solid density = fluid density**, return **SUSPENDED**.
7. Display the corresponding result in main().
8. Stop.

According to question , the constants are:

FLOATS = -1, SUSPENDED = 0, and SINKS = 1.

Algorithm 17 — Calculate Square Root Using Newton-Raphson Method

Function: calculate_square_root(double number)

1. Start.
2. Receive number.
3. If number < 0, return **-1**.
4. If number == 0, return **0**.
5. Set:

old_value = number

6. Using a **do-while loop**, calculate:

new_value = $0.5 \times (\text{old_value} + \text{number} / \text{old_value})$

7. Calculate:

difference = new_value - old_value

8. If difference is negative, convert it to a positive value.
9. Set:

old_value = new_value

10. Repeat the loop while:

difference >= TOLERANCE

11. Return new_value as the approximate square root.
12. Stop.

Important: Do not use sqrt(), fabs(), or any function from math.h. The project requires the **Newton-Raphson method**, with TOLERANCE = 0.000001.

Test Case	Type	Operation	Solid	Input	Expected Output
1	Valid	Surface Area	Sphere	Radius = 2 m	50.2656 m ²

Calculation

Formula:

$$\text{Surface Area} = 4 \times \text{PI} \times r \times r$$

Given:

$$r = 2 \text{ m}$$

$$\text{PI} = 3.1416$$

Calculation:

$$= 4 \times 3.1416 \times 2 \times 2$$

$$= \mathbf{50.2656 \text{ m}^2}$$

Expected Output:

$$\text{Surface Area} = 50.2656 \text{ m}^2$$

Test Case 2 — Valid Input: Cylinder Volume

Test Case	Type	Operation	Solid	Input	Expected Output
2	Valid	Volume	Cylinder	Radius = 2 m, Height = 5 m	62.8320 m ³

Calculation

Formula:

$$\text{Volume} = \text{PI} \times \text{radius} \times \text{radius} \times \text{height}$$

Given:

$$\text{Radius} = 2 \text{ m}$$

$$\text{Height} = 5 \text{ m}$$

$$\text{PI} = 3.1416$$

Calculation:

$$= 3.1416 \times 2 \times 2 \times 5$$

$$= 3.1416 \times 20$$

$$= \mathbf{62.8320 \text{ m}^3}$$

Expected Output:

$$\text{Volume} = 62.8320 \text{ m}^3$$

Test Case 3 — Valid Input: Cone Surface Area

Test Case	Type	Operation	Solid	Input	Expected Output
3		Valid Surface Area	Cone	Radius = 3 m, Height = 4 m	75.3984 m ²

Calculation

First calculate the **slant height** using the square-root function,

Slant Height:

$$\begin{aligned} &= \sqrt{\text{radius}^2 + \text{height}^2} \\ &= \sqrt{3^2 + 4^2} \\ &= \sqrt{25} \\ &= 5 \text{ m} \end{aligned}$$

Formula:

$$\text{Surface Area} = \text{PI} \times \text{radius} \times (\text{radius} + \text{slant height})$$

Calculation:

$$\begin{aligned} &= 3.1416 \times 3 \times (3 + 5) \\ &= 3.1416 \times 3 \times 8 \\ &= 75.3984 \text{ m}^2 \end{aligned}$$

Expected Output:

$$\text{Surface Area} = 75.3984 \text{ m}^2$$

Test Case 4 — Valid Input: Cuboid Base Area

Test Case	Type	Operation	Solid	Input	Expected Output
4		Valid Base Area	Cuboid	Length = 5 m, Breadth = 3 m	15.0000 m ²

Calculation

Formula:

$$\text{Base Area} = \text{Length} \times \text{Breadth}$$

Given:

$$\begin{aligned} \text{Length} &= 5 \text{ m} \\ \text{Breadth} &= 3 \text{ m} \end{aligned}$$

Calculation:

$$\begin{aligned} &= 5 \times 3 \\ &= 15 \text{ m}^2 \end{aligned}$$

Expected Output:

$$\text{Base Area} = 15.0000 \text{ m}^2$$

It specifies that `cuboid_base_area()` returns `length × breadth`.

Test Case 5 — Valid Input: Calculate Mass of a Sphere

Test Case	Type	Operation	Solid	Input	Expected Output
5	Valid	Mass	Sphere	Radius = 2 m, Density = 500 kg/m ³	16755.2 kg

Calculation

First calculate the **volume of the sphere**.

Formula:

$$\text{Volume} = (4/3) \times \text{PI} \times r^3$$

Given:

Radius = 2 m

Density = 500 kg/m³

PI = 3.1416

Volume:

$$= (4/3) \times 3.1416 \times 2 \times 2 \times 2$$

$$= \mathbf{33.5104 \text{ m}^3}$$

Now calculate mass:

$$\mathbf{\text{Mass} = \text{Density} \times \text{Volume}}$$

$$= 500 \times 33.5104$$

$$= \mathbf{16755.2 \text{ kg}}$$

Expected Output:

Mass = 16755.2000 kg

It requires calculate_mass() to calculate the appropriate solid's volume first and then use **mass = density × volume**.

Test Case 6 — Valid Input: Calculate Weight of a Cylinder

Test Case	Type	Operation	Solid	Input	Expected Output
6	Valid	Weight	Cylinder	Radius = 1 m, Height = 2 m, Density = 500 kg/m ³	30787.68 N

Calculation

First calculate the **volume of cylinder**:

$$\mathbf{\text{Volume} = \text{PI} \times r \times r \times h}$$

$$= 3.1416 \times 1 \times 1 \times 2$$

$$= \mathbf{6.2832 \text{ m}^3}$$

Now calculate **mass**:

$$\text{Mass} = \text{Density} \times \text{Volume}$$

$$= 500 \times 6.2832$$

$$= \mathbf{3141.6 \text{ kg}}$$

Now calculate **weight**:

$$\text{Weight} = \text{Mass} \times \text{GRAVITY}$$

$$= 3141.6 \times 9.8$$

$$= \mathbf{30787.68 \text{ N}}$$

Expected Output:

Weight = 30787.6800 N

It requires calculate_weight() to obtain the mass through calculate_mass() and then use **weight = mass × GRAVITY**.

Test Case 7 — Valid Input: Gravitational Potential Energy of a Cuboid

Test Case	Type	Operation	Solid	Input	Expected Output
7	Valid	Potential Energy	Cuboid	L = 2 m, B = 1 m, H = 1 m, Density = 500 kg/m ³ , Elevation = 10 m	98000 J

Calculation

First calculate the **volume of cuboid**:

$$\text{Volume} = \text{Length} \times \text{Breadth} \times \text{Height}$$

$$= 2 \times 1 \times 1$$

$$= \mathbf{2 \text{ m}^3}$$

Now calculate **mass**:

$$\text{Mass} = \text{Density} \times \text{Volume}$$

$$= 500 \times 2$$

$$= \mathbf{1000 \text{ kg}}$$

Now calculate **Potential Energy**:

$$\text{P.E.} = \text{Mass} \times \text{GRAVITY} \times \text{Elevation}$$

$$= 1000 \times 9.8 \times 10$$

$$= \mathbf{98000 \text{ J}}$$

Expected Output:

Potential Energy = 98000.0000 J

It specifies that potential_energy() obtains mass from calculate_mass() and calculates **mass × GRAVITY × elevation**.

Test Case 8 — Valid Input: Pressure on the Base of a Cylinder

Test Case	Type	Operation	Solid	Input	Expected Output
8	Valid	Pressure	Cylinder	Radius = 1 m, Height = 2 m, Density = 500 kg/m ³	9800 Pa

Calculation

First calculate the **volume of cylinder**:

$$\text{Volume} = \text{PI} \times r \times r \times h$$

$$= 3.1416 \times 1 \times 1 \times 2$$

$$= \mathbf{6.2832 \text{ m}^3}$$

Now calculate the **contact/base area**:

$$\text{Base Area} = \text{PI} \times r \times r$$

$$= 3.1416 \times 1 \times 1$$

$$= \mathbf{3.1416 \text{ m}^2}$$

Now calculate pressure:

$$\text{Pressure} = (\text{Density} \times \text{Volume} \times \text{GRAVITY}) / \text{Contact Area}$$

$$= (500 \times 6.2832 \times 9.8) / 3.1416$$

$$= \mathbf{9800 \text{ Pa}}$$

Expected Output:

Pressure = 9800.0000 Pa

This follows the required pressure formula and uses the circular base area for a cylinder.

Test Case 9 — Valid Input: Buoyant Force on a Sphere

Test Case	Type	Operation	Solid	Input	Expected Output
9	Valid	Buoyant Force	Sphere	Radius = 1 m, Fluid Density = 1000 kg/m ³	41050.24 N

Calculation

First calculate the **volume of the sphere**:

$$\text{Volume} = (4/3) \times \text{PI} \times r^3$$

$$= (4/3) \times 3.1416 \times 1 \times 1 \times 1$$

$$= \mathbf{4.1888 \text{ m}^3}$$

Now calculate the **buoyant force**:

$$\text{Buoyant Force} = \text{Fluid Density} \times \text{Volume} \times \text{GRAVITY}$$

$$= 1000 \times 4.1888 \times 9.8$$

$$= \mathbf{41050.24 \text{ N}}$$

Expected Output:

Buoyant Force = 41050.2400 N

It requires `buoyant_force()` to calculate the selected solid's volume and then use **fluid density × volume × GRAVITY**.

Test Case 10 — Valid Input: Buoyancy Behaviour

Test Case	Type	Operation	Solid Density	Fluid Density	Expected Output
-----------	------	-----------	---------------	---------------	-----------------

10		Valid Buoyancy Behaviour	800 kg/m ³	1000 kg/m ³	FLOATS
----	--	--------------------------	-----------------------	------------------------	--------

Calculation / Comparison**Given:**

Solid Density = **800 kg/m³**

Fluid Density = **1000 kg/m³**

Compare the densities:

800 < 1000

Therefore:

Solid Density < Fluid Density → FLOATS

Expected Output:

The solid floats.

It specifies: if solid density is **less than** fluid density, return FLOATS; if greater, return SINKS; and if equal, return SUSPENDED.

Test Case 11 — Invalid Input: Negative Radius

Test Case	Type	Operation	Solid	Input	Expected Output
-----------	------	-----------	-------	-------	-----------------

11		Invalid Surface Area Sphere	Radius = -2 m	Error message and ask for radius again
----	--	-----------------------------	---------------	--

Validation / Calculation**Given:**

Radius = **-2 m**

According to the project:

Radius must be > 0

Check:

-2 ≤ 0 → Invalid Input

Therefore, **no surface-area calculation is performed**.

Expected Output:

Invalid radius! Radius must be greater than zero.

Enter radius again:

For example, if the user then enters **2 m**, the program accepts it and calculates:

Surface Area
 $= 4 \times 3.1416 \times 2 \times 2$
= 50.2656 m²

It requires dimensions to be greater than zero; when an invalid value is entered, an error message must be displayed and the value requested again.

Test Case 12 — Invalid Input: Zero Density

Test Case Type	Operation Solid	Input	Expected Output
12	Invalid Mass	Cylinder Density = 0 kg/m³	Error message and ask for density again

Validation / Calculation

Given:

Density = **0 kg/m³**

According to the project:

Density must be > 0

Check:

$0 \leq 0 \rightarrow$ Invalid Input

Therefore, the program **does not calculate mass** and asks the user to enter the density again.

Expected Output:

Invalid density! Density must be greater than zero.

Enter density again:

There is **no numerical calculation for this invalid value** because it must first be rejected. It requires all densities to be greater than zero and invalid values to be requested again.

Test Case 12 — Invalid Input: Zero Density

Test Case Type	Operation Solid	Input	Expected Output
12	Invalid Mass	Cylinder Density = 0 kg/m³	Error message and ask for density again

Validation / Calculation

Given:

Density = **0 kg/m³**

According to the project:

Density must be > 0

Check:

$0 \leq 0 \rightarrow$ Invalid Input

Therefore, the program **does not calculate mass** and asks the user to enter the density again.

Expected Output:

Invalid density! Density must be greater than zero.

Enter density again:

There is **no numerical calculation for this invalid value** because it must first be rejected. It requires all densities to be greater than zero and invalid values to be requested again.

Test Case 13 — Invalid Input: Negative Elevation

Test Case	Type	Operation	Input	Expected Output
13	Invalid	Gravitational Potential Energy	Elevation = -5 m	Error message and ask for elevation again

Validation / Calculation

Given:

Elevation = **-5 m**

According to the project:

Elevation must not be negative.

Check:

$-5 < 0 \rightarrow$ Invalid Input

Therefore, the program **does not calculate potential energy** and asks for elevation again.

Expected Output:

Invalid elevation! Elevation cannot be negative.

Enter elevation again:

Note: Unlike dimensions and densities, **elevation = 0 is valid** because the project says elevation must **not be negative**.

Test Case 14 — Boundary Input: Elevation = 0

Test Case Type	Operation	Input	Expected Output
14	Boundary Gravitational Potential Energy	Elevation = 0 m	Potential Energy = 0 J

Calculation

The project allows **zero elevation** because elevation must be **non-negative (≥ 0)**.

Formula:

Potential Energy = Mass $\times$ GRAVITY $\times$ Elevation

Given:

Elevation = 0 m

Calculation:

= Mass $\times$ 9.8 $\times$ 0

= **0 J**

So regardless of the valid mass:

Potential Energy = 0 J

Expected Output

Potential Energy = 0.0000 J

This is an important **boundary test case** because 0 is the lowest valid value for elevation.

Test Case 15 — Boundary Input: Very Small Positive Radius

Test Case Type	Operation	Solid	Input	Expected Output
15	Boundary Volume	Sphere	Radius = 0.1 m	Volume = 0.0041888 m³

Calculation

The project requires dimensions to be **greater than zero**, so 0.1 m is a valid small positive test value.

Formula:

Volume = $(4/3) \times \text{PI} \times r^3$

Given:

$r = 0.1 \text{ m}$

$\pi = 3.1416$

Calculation:

$$= \left(\frac{4}{3}\right) \times 3.1416 \times 0.1 \times 0.1 \times 0.1$$

$$= \left(\frac{4}{3}\right) \times 3.1416 \times 0.001$$

$$= \mathbf{0.0041888 \text{ m}^3}$$

Expected Output

Volume = 0.0042 m³ (*if displayed to 4 decimal places*)

Test Case 16 — Invalid Input: Invalid Solid Choice

Test Case Type	Operation Input	Expected Output
16	Invalid Volume Solid Choice = 5	Error message and ask for solid choice again

Validation

The valid solid choices are:

1 = Sphere

2 = Cylinder

3 = Cone

4 = Cuboid

Given:

Solid Choice = 5

Check:

5 > 4 → Invalid Choice

Therefore, the program should **not perform any volume calculation**.

Expected Output

Invalid solid choice! Please enter a value from 1 to 4.

Then the program asks for the solid choice again.

It specifically requires solid choices to be within the specified range and invalid inputs to be requested again.

Test Case 17 — Invalid Operation: Sphere Selected for Base Area

Test Case Type	Operation	Solid	Input	Expected Output
17	Invalid Base Area	Sphere	Solid Choice = 1	Display message and return to main menu

Validation

The project states that **Base Area** is available only for:

- Cylinder
- Cone
- Cuboid

A sphere does **not** have an ideal geometrical base area defined by the given dimensions.

Given:

Solid Choice = **1 (Sphere)**

Therefore:

Sphere + Base Area → Not Allowed

Calculation

No calculation is performed.

Expected Output

Base area cannot be calculated for a sphere.

Then the program should **return to the main menu**.

Test Case 18 — Boundary Case: Suspended Buoyancy

Test Case Type	Operation	Solid Density	Fluid Density	Expected Output
18	Boundary Buoyancy Behaviour	1000 kg/m ³	1000 kg/m ³	SUSPENDED

Calculation / Comparison

Given:

Solid Density = **1000 kg/m³**

Fluid Density = **1000 kg/m³**

Compare:

1000 = 1000

According to the project:

Solid Density = Fluid Density → SUSPENDED

Expected Output

The solid remains suspended.

The function `determine_buoyancy()` should return **SUSPENDED = 0** in this case.

Test Case 19 — Valid Case: Sinking Behaviour

Test Case	Type	Operation	Solid Density	Fluid Density	Expected Output
19	Valid Buoyancy Behaviour		1200 kg/m ³	1000 kg/m ³	SINKS

Calculation / Comparison

Given:

Solid Density = 1200 kg/m³

Fluid Density = 1000 kg/m³

Compare:

1200 > 1000

Therefore:

Solid Density > Fluid Density → SINKS

Expected Output

The solid sinks.

The function `determine_buoyancy()` should return **SINKS = 1**.

Test Case 20 — Valid Case: Floating Behaviour

Test Case	Type	Operation	Solid Density	Fluid Density	Expected Output
20	Valid Buoyancy Behaviour		800 kg/m ³	1000 kg/m ³	FLOATS

Calculation / Comparison

Given:

Solid Density = 800 kg/m³

Fluid Density = 1000 kg/m³

Compare:

800 < 1000

Therefore:

Solid Density < Fluid Density → FLOATS

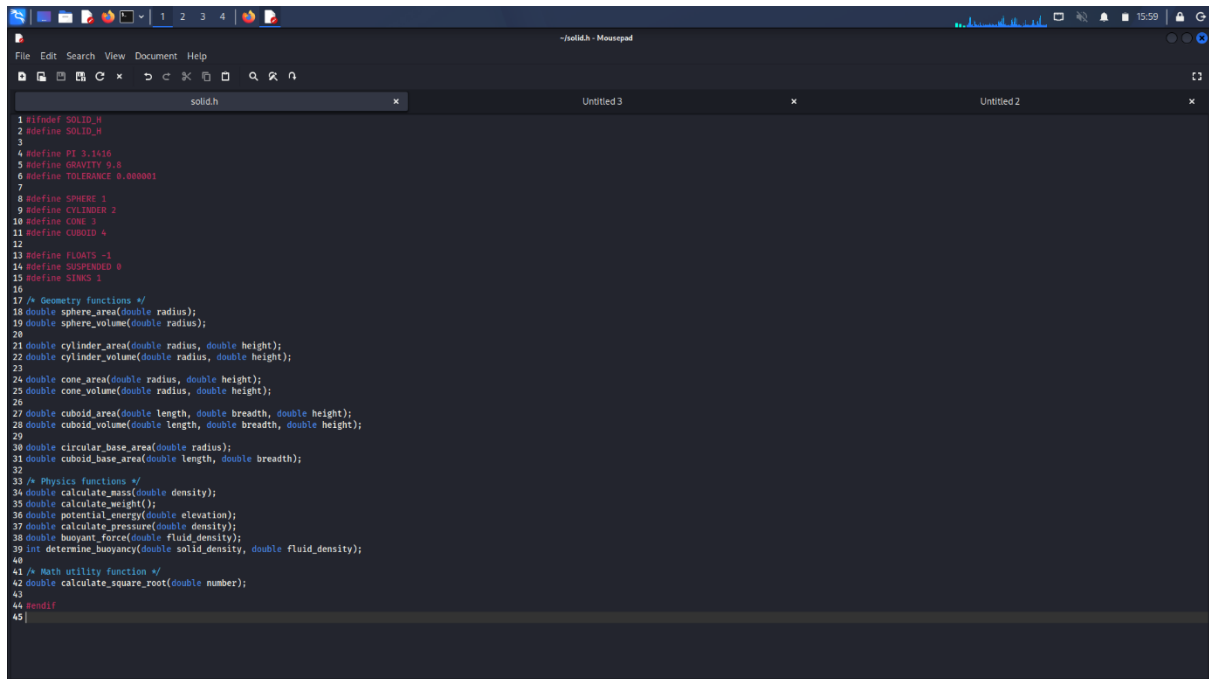
Expected Output

The solid floats.

The function `determine_buoyancy()` returns **FLOATS = -1**.

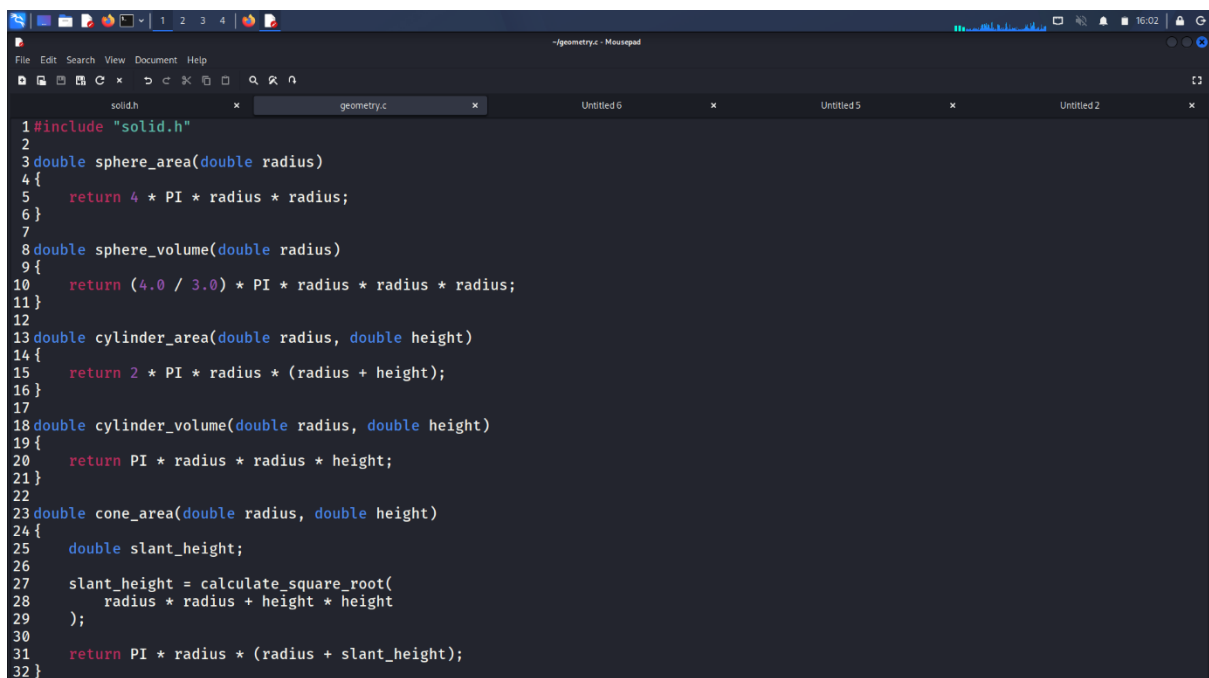
>Write a code C.

File 1:

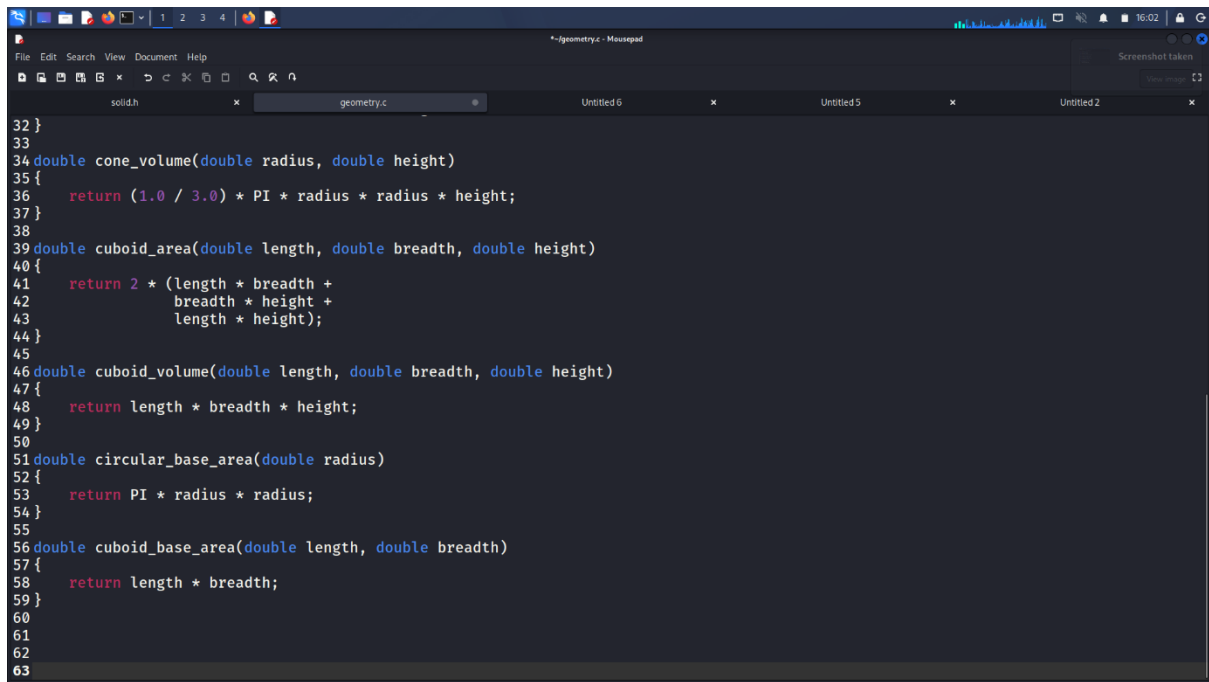


```
1 #ifndef SOLID_H
2 #define SOLID_H
3
4 #define PI 3.14159
5 #define GRAVITY 9.8
6 #define TOLERANCE 0.000001
7
8 #define SPHERE 1
9 #define CYLINDER 2
10 #define CONE 3
11 #define CUBOID 4
12
13 #define FLOATS -1
14 #define SUSPENDED 0
15 #define SINKS 1
16
17 /* Geometry functions */
18 double sphere_area(double radius);
19 double sphere_volume(double radius);
20
21 double cylinder_area(double radius, double height);
22 double cylinder_volume(double radius, double height);
23
24 double cone_area(double radius, double height);
25 double cone_volume(double radius, double height);
26
27 double cuboid_area(double length, double breadth, double height);
28 double cuboid_volume(double length, double breadth, double height);
29
30 double circular_base_area(double radius);
31 double cuboid_base_area(double length, double breadth);
32
33 /* Physics functions */
34 double calculate_mass(double density);
35 double calculate_weight();
36 double potential_energy(double elevation);
37 double calculate_pressure(double density);
38 double buoyant_force(double fluid_density);
39 int determine_buoyancy(double solid_density, double fluid_density);
40
41 /* Math utility function */
42 double calculate_square_root(double number);
43
44 #endif
45
```

File 2:



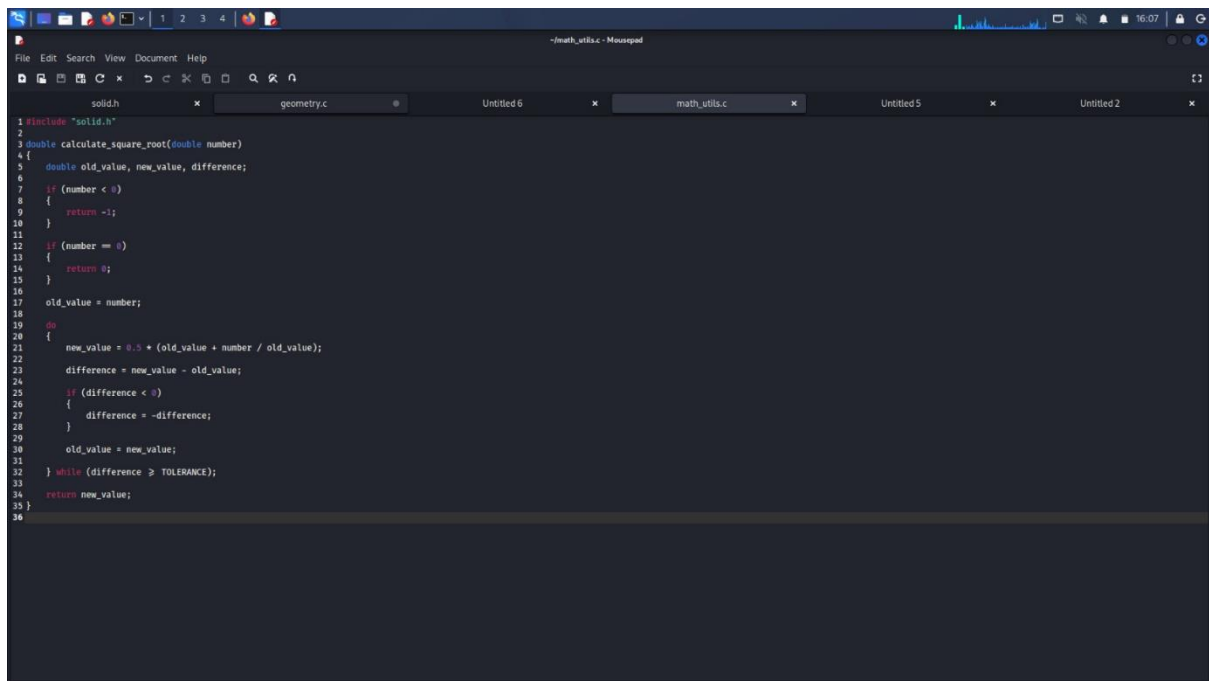
```
1 #include "solid.h"
2
3 double sphere_area(double radius)
4 {
5     return 4 * PI * radius * radius;
6 }
7
8 double sphere_volume(double radius)
9 {
10     return (4.0 / 3.0) * PI * radius * radius * radius;
11 }
12
13 double cylinder_area(double radius, double height)
14 {
15     return 2 * PI * radius * (radius + height);
16 }
17
18 double cylinder_volume(double radius, double height)
19 {
20     return PI * radius * radius * height;
21 }
22
23 double cone_area(double radius, double height)
24 {
25     double slant_height;
26
27     slant_height = calculate_square_root(
28         radius * radius + height * height
29     );
30
31     return PI * radius * (radius + slant_height);
32 }
```



A screenshot of a code editor window titled "geometry.c - Mousepad". The editor shows the implementation of several geometric functions in C. The code is as follows:

```
32 }
33
34 double cone_volume(double radius, double height)
35 {
36     return (1.0 / 3.0) * PI * radius * radius * height;
37 }
38
39 double cuboid_area(double length, double breadth, double height)
40 {
41     return 2 * (length * breadth +
42                breadth * height +
43                length * height);
44 }
45
46 double cuboid_volume(double length, double breadth, double height)
47 {
48     return length * breadth * height;
49 }
50
51 double circular_base_area(double radius)
52 {
53     return PI * radius * radius;
54 }
55
56 double cuboid_base_area(double length, double breadth)
57 {
58     return length * breadth;
59 }
60
61
62
63
```

>File 3:



A screenshot of a code editor window titled "math_utils.c - Mousepad". The editor shows the implementation of a function to calculate the square root of a number using the Newton-Raphson method. The code is as follows:

```
1 #include "solid.h"
2
3 double calculate_square_root(double number)
4 {
5     double old_value, new_value, difference;
6
7     if (number < 0)
8     {
9         return -1;
10    }
11
12    if (number == 0)
13    {
14        return 0;
15    }
16
17    old_value = number;
18
19    do
20    {
21        new_value = 0.5 * (old_value + number / old_value);
22        difference = new_value - old_value;
23
24        if (difference < 0)
25        {
26            difference = -difference;
27        }
28
29        old_value = new_value;
30    } while (difference > TOLERANCE);
31
32    return new_value;
33 }
34
35
```

>File 4:

```
1 #include <stdio.h>
2 #include "solid.h"
3
4 int main()
5 {
6     int choice, solid, result;
7     double radius, height, length, breadth;
8     double density, fluid_density, solid_density;
9     double elevation, answer;
10
11     do
12     {
13         printf("===== SOLID AND PHYSICS MENU =====\n");
14         printf("1. Surface Area\n");
15         printf("2. Volume\n");
16         printf("3. Base Area\n");
17         printf("4. Mass\n");
18         printf("5. Weight\n");
19         printf("6. Gravitational Potential Energy\n");
20         printf("7. Pressure on the Base\n");
21         printf("8. Buoyant Force\n");
22         printf("9. Buoyancy Behaviour\n");
23         printf("10. Exit\n");
24
25         printf("Enter your choice: ");
26         scanf("%d", &choice);
27
28         switch (choice)
29         {
30             case 1:
31                 do
32                 {
33                     printf("1.Sphere 2.Cylinder 3.Cone 4.Cuboid: ");
34                     scanf("%d", &solid);
35                     while (solid < 1 || solid > 4);
36
37                     if (solid == SPHERE)
38                     {
39                         do
40                         {
41                             printf("Enter radius: ");
42                             scanf("%lf", &radius);
43                             while (radius <= 0);
44
45                             answer = sphere_area(radius);
46
47                         } else if (solid == CYLINDER)
48                         {
49                             do
50                             {
```

```
50         {
51             printf("Enter radius: ");
52             scanf("%lf", &radius);
53             while (radius <= 0);
54
55             do
56             {
57                 printf("Enter height: ");
58                 scanf("%lf", &height);
59                 while (height <= 0);
60
61                 answer = cylinder_area(radius, height);
62
63             } else if (solid == CONE)
64             {
65                 do
66                 {
67                     printf("Enter radius: ");
68                     scanf("%lf", &radius);
69                     while (radius <= 0);
70
71                     do
72                     {
73                         printf("Enter height: ");
74                         scanf("%lf", &height);
75                         while (height <= 0);
76
77                         answer = cone_area(radius, height);
78                     }
79                 } else
80                 {
81                     do
82                     {
83                         printf("Enter length: ");
84                         scanf("%lf", &length);
85                         while (length <= 0);
86
87                     }
88                     do
89                     {
90                         printf("Enter breadth: ");
91                         scanf("%lf", &breadth);
92                         while (breadth <= 0);
93
94                     }
95                     do
96                     {
97                         printf("Enter height: ");
98                         scanf("%lf", &height);
99                         while (height <= 0);
100
101                     }
102                     answer = cuboid_area(length, breadth, height);
103                 }
```

```

100     }
101
102     printf("Surface Area = %.4f square metres\n", answer);
103     break;
104
105     case 2:
106     do
107     {
108         printf("1.Sphere 2.Cylinder 3.Cone 4.Cuboid: ");
109         scanf("%d", &solid);
110     } while (solid < 1 || solid > 4);
111
112     if (solid == SPHERE)
113     {
114         do
115         {
116             printf("Enter radius: ");
117             scanf("%d", &radius);
118         } while (radius <= 0);
119
120         answer = sphere_volume(radius);
121     }
122     else if (solid == CYLINDER)
123     {
124         do
125         {
126             printf("Enter radius: ");
127             scanf("%d", &radius);
128         } while (radius <= 0);
129
130         do
131         {
132             printf("Enter height: ");
133             scanf("%d", &height);
134         } while (height <= 0);
135
136         answer = cylinder_volume(radius, height);
137     }
138     else if (solid == CONE)
139     {
140         do
141         {
142             printf("Enter radius: ");
143             scanf("%d", &radius);
144         } while (radius <= 0);
145
146         do
147         {
148             printf("Enter height: ");
149             scanf("%d", &height);

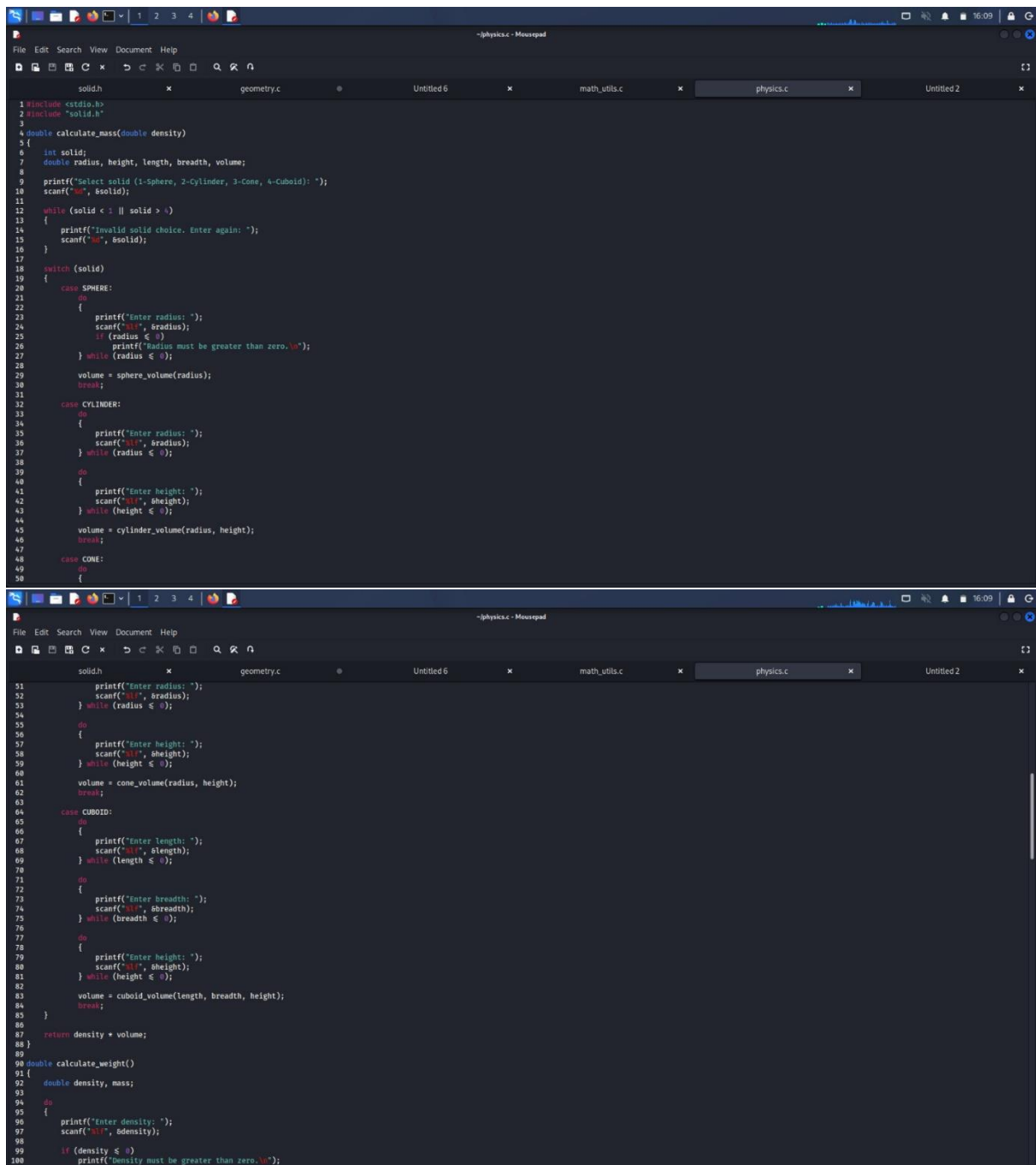
```

```

150     } while (height <= 0);
151
152     answer = cone_volume(radius, height);
153 }
154 else
155 {
156     do
157     {
158         printf("Enter length: ");
159         scanf("%d", &length);
160     } while (length <= 0);
161
162     do
163     {
164         printf("Enter breadth: ");
165         scanf("%d", &breadth);
166     } while (breadth <= 0);
167
168     do
169     {
170         printf("Enter height: ");
171         scanf("%d", &height);
172     } while (height <= 0);
173
174     answer = cuboid_volume(length, breadth, height);
175 }
176
177 printf("Volume = %.4f cubic metres\n", answer);
178 break;
179
180 case 3:
181 printf("Select solid: 1.Cylinder 2.Cone 3.Cuboid\n");
182 printf("Enter choice: ");
183 scanf("%d", &solid);
184
185 if (solid == SPHERE)
186 {
187     printf("Base area cannot be calculated for a sphere.\n");
188     break;
189 }
190
191 while (solid < CYLINDER || solid > CUBOID)
192 {
193     printf("Invalid choice. Enter 1, 2 or 3: ");
194     scanf("%d", &solid);
195 }
196
197 if (solid == CYLINDER || solid == CONE)
198 {
199     do

```

>File 5:



```
1#include <stdio.h>
2#include "solid.h"
3
4double calculate_mass(double density)
5{
6    int solid;
7    double radius, height, length, breadth, volume;
8
9    printf("Select solid (1-Sphere, 2-Cylinder, 3-Cone, 4-Cuboid): ");
10   scanf("%d", &solid);
11
12   while (solid < 1 || solid > 4)
13   {
14       printf("Invalid solid choice. Enter again: ");
15       scanf("%d", &solid);
16   }
17
18   switch (solid)
19   {
20       case SPHERE:
21       do
22       {
23           printf("Enter radius: ");
24           scanf("%lf", &radius);
25           if (radius <= 0)
26               printf("Radius must be greater than zero.\n");
27           } while (radius <= 0);
28       volume = sphere_volume(radius);
29       break;
30       case CYLINDER:
31       do
32       {
33           printf("Enter radius: ");
34           scanf("%lf", &radius);
35           } while (radius <= 0);
36       do
37       {
38           printf("Enter height: ");
39           scanf("%lf", &height);
40           } while (height <= 0);
41       volume = cylinder_volume(radius, height);
42       break;
43       case CONE:
44       do
45       {
46           printf("Enter radius: ");
47           scanf("%lf", &radius);
48           } while (radius <= 0);
49       do
50       {
51           printf("Enter height: ");
52           scanf("%lf", &height);
53           } while (height <= 0);
54       volume = cone_volume(radius, height);
55       break;
56       case CUBOID:
57       do
58       {
59           printf("Enter length: ");
60           scanf("%lf", &length);
61           } while (length <= 0);
62       do
63       {
64           printf("Enter breadth: ");
65           scanf("%lf", &breadth);
66           } while (breadth <= 0);
67       do
68       {
69           printf("Enter height: ");
70           scanf("%lf", &height);
71           } while (height <= 0);
72       volume = cuboid_volume(length, breadth, height);
73       break;
74   }
75   return density * volume;
76 }
77
78double calculate_weight()
79{
80    double density, mass;
81
82    printf("Enter density: ");
83    scanf("%lf", &density);
84    if (density <= 0)
85        printf("Density must be greater than zero.\n");
86}
```

```
101 } while (density <= 0);
102
103 mass = calculate_mass(density);
104
105 return mass * GRAVITY;
106 }
107 }
108 }
109 double potential_energy(double elevation)
110 {
111     double density, mass;
112
113     do
114     {
115         printf("Enter density: ");
116         scanf("%lf", &density);
117         if (density <= 0)
118             printf("Density must be greater than zero.\n");
119     } while (density <= 0);
120
121 mass = calculate_mass(density);
122
123 return mass * GRAVITY * elevation;
124 }
125 }
126 }
127
128 double calculate_pressure(double density)
129 {
130     int solid;
131     double radius, height, length, breadth;
132     double volume, contact_area;
133
134     do
135     {
136         printf("Select solid (2-Cylinder, 3-Cone, 4-Cuboid): ");
137         scanf("%d", &solid);
138
139         if (solid < CYLINDER || solid > CUBOID)
140             printf("Invalid choice. Select Cylinder, Cone or Cuboid.\n");
141     } while (solid < CYLINDER || solid > CUBOID);
142
143 if (solid == CYLINDER || solid == CONE)
144 {
145     do
146     {
147         printf("Enter radius: ");
148         scanf("%lf", &radius);
149     } while (radius <= 0);
150
151     do
152     {
153         printf("Enter height: ");
154         scanf("%lf", &height);
155     } while (height <= 0);
156
157     contact_area = circular_base_area(radius);
158
159     if (solid == CYLINDER)
160         volume = cylinder_volume(radius, height);
161     else
162         volume = cone_volume(radius, height);
163 }
164 else
165 {
166     do
167     {
168         printf("Enter length: ");
169         scanf("%lf", &length);
170     } while (length <= 0);
171
172     do
173     {
174         printf("Enter breadth: ");
175         scanf("%lf", &breadth);
176     } while (breadth <= 0);
177
178     do
179     {
180         printf("Enter height: ");
181         scanf("%lf", &height);
182     } while (height <= 0);
183
184     contact_area = cuboid_base_area(length, breadth);
185     volume = cuboid_volume(length, breadth, height);
186 }
187
188 return (density * volume * GRAVITY) / contact_area;
189 }
190 }
191
192 double buoyant_force(double fluid_density)
193 {
194     int solid;
195     double radius, height, length, breadth, volume;
196
197     do
198     {
199         printf("Select solid (1-Sphere, 2-Cylinder, 3-Cone, 4-Cuboid): ");
200         scanf("%d", &solid);
```

```
201 } while (solid < SPHERE || solid > CUBOID);
202
203 switch (solid)
204 {
205     case SPHERE:
206     do
207     {
208         printf("Enter radius: ");
209         scanf("%lf", &radius);
210     } while (radius <= 0);
211
212     volume = sphere_volume(radius);
213     break;
214
215     case CYLINDER:
216     case CONE:
217     do
218     {
219         printf("Enter radius: ");
220         scanf("%lf", &radius);
221     } while (radius <= 0);
222
223     do
224     {
225         printf("Enter height: ");
226         scanf("%lf", &height);
227     } while (height <= 0);
228
229     if (solid == CYLINDER)
230         volume = cylinder_volume(radius, height);
231     else
232         volume = cone_volume(radius, height);
233     break;
234
235     case CUBOID:
236     do
237     {
238         printf("Enter length: ");
239         scanf("%lf", &length);
240     } while (length <= 0);
241
242     do
243     {
244         printf("Enter breadth: ");
245         scanf("%lf", &breadth);
246     } while (breadth <= 0);
247
248     do
249     {
250         printf("Enter height: ");
```

```
249     {
250         printf("Enter height: ");
251         scanf("%lf", &height);
252     } while (height <= 0);
253
254     volume = cuboid_volume(length, breadth, height);
255     break;
256 }
257
258 return fluid_density * volume * GRAVITY;
259 }
260
261 int determine_buoyancy(double solid_density, double fluid_density)
262 {
263     if (solid_density > fluid_density)
264         return SINKS;
265     else if (solid_density < fluid_density)
266         return FLOATS;
267     else
268         return SUSPENDED;
269 }
270
```

>Compilation and Execution of test cases.

*Test case 1:

```
Kali Linux
Session Actions Edit View Help
zsh: corrupt history file /home/priyadarshna/.zsh_history
(priyadarshna@kali) ~
$ gcc main.c geometry.c physics.c math_utils.c -o solid_projects
(priyadarshna@kali) ~
$ ./solid_projects

===== SOLID AND PHYSICS MENU =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: 1
1.Sphere 2.Cylinder 3.Cone 4.Cuboid: 1
Enter radius: 2
Surface Area = 58.2656 square metres

===== SOLID AND PHYSICS MENU =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: 
```

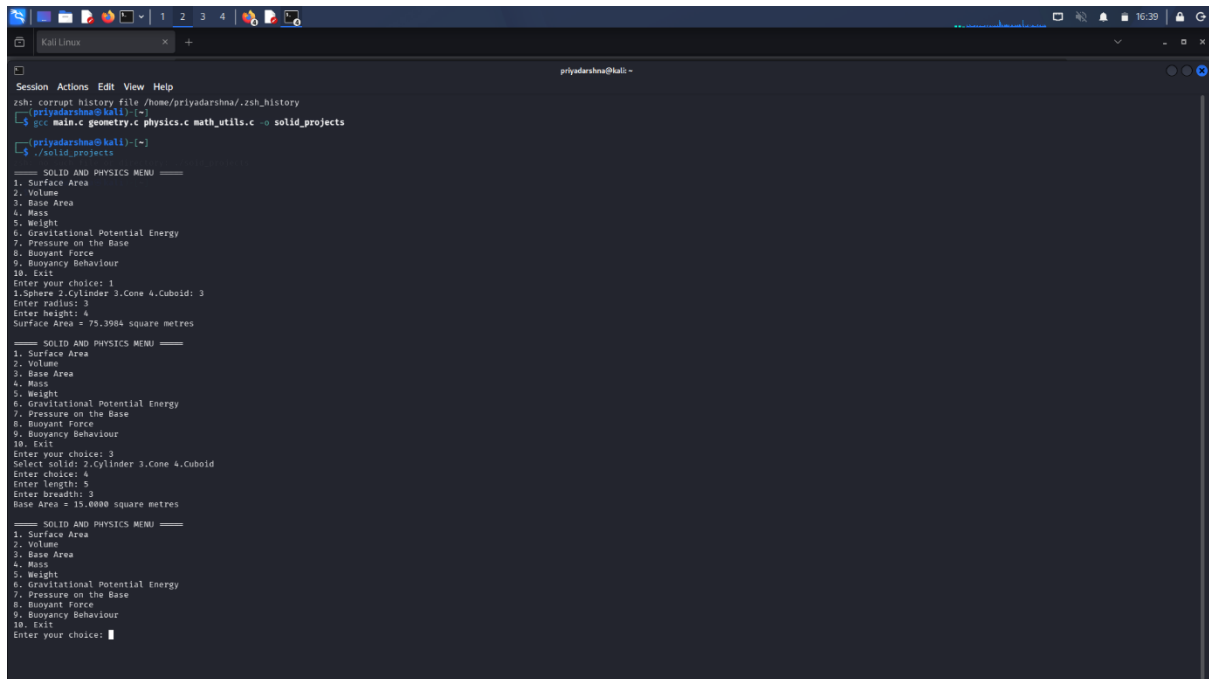
*Test case 2:

```
Kali Linux
Session Actions Edit View Help
zsh: corrupt history file /home/priyadarshna/.zsh_history
(priyadarshna@kali) ~
$ gcc main.c geometry.c physics.c math_utils.c -o solid_projects
(priyadarshna@kali) ~
$ ./solid_projects

===== SOLID AND PHYSICS MENU =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: 1
1.Sphere 2.Cylinder 3.Cone 4.Cuboid: 3
Enter radius: 3
Enter height: 4
Surface Area = 75.3984 square metres

===== SOLID AND PHYSICS MENU =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: 
```

*Test case -3,4,5 :



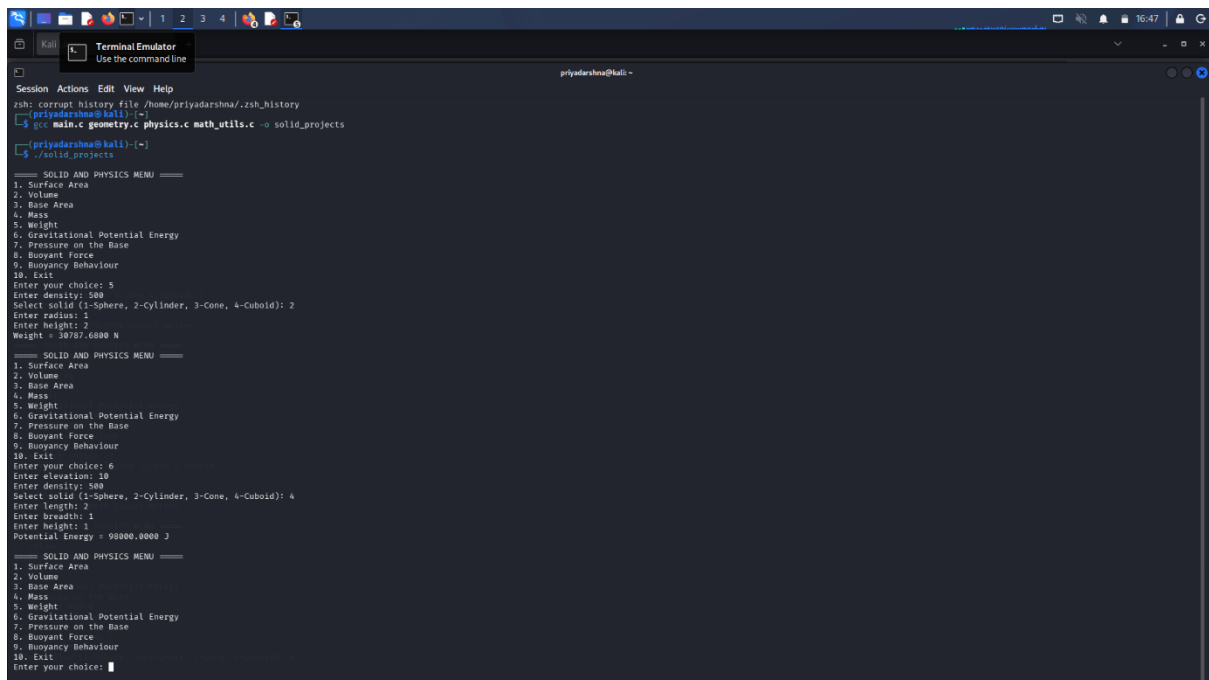
```
zsh: corrupt history file /home/priyadarshma/.zsh_history
(priyadarshma@kali) ~-
$ gcc main.c geometry.c physics.c math_utils.c -o solid_projects
(priyadarshma@kali) ~-
$ ./solid_projects

===== SOLID AND PHYSICS MENU =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: 1
1.Sphere 2.Cylinder 3.Cone 4.Cuboid: 3
Enter radius: 3
Enter height: 4
Surface Area = 75.3984 square metres

===== SOLID AND PHYSICS MENU =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: 3
Select solid: 2.Cylinder 3.Cone 4.Cuboid: 3
Enter choice: 4
Enter length: 5
Enter breadth: 2
Base Area = 15.0000 square metres

===== SOLID AND PHYSICS MENU =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: █
```

*Test Case -6,7:



```
Kali Terminal Emulator
Use the command line

zsh: corrupt history file /home/priyadarshma/.zsh_history
(priyadarshma@kali) ~-
$ gcc main.c geometry.c physics.c math_utils.c -o solid_projects
(priyadarshma@kali) ~-
$ ./solid_projects

===== SOLID AND PHYSICS MENU =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: 5
Enter density: 500
Select solid (1-Sphere, 2-Cylinder, 3-Cone, 4-Cuboid): 2
Enter radius: 1
Enter height: 2
Weight = 30787.6800 N

===== SOLID AND PHYSICS MENU =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: 6
Enter elevation: 10
Enter density: 500
Select solid (1-Sphere, 2-Cylinder, 3-Cone, 4-Cuboid): 4
Enter length: 2
Enter breadth: 1
Enter height: 1
Potential Energy = 90000.0000 J

===== SOLID AND PHYSICS MENU =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: █
```

*Test Case -8,9,10

```
Kali Linux
Session Actions Edit View Help
zsh: corrupt history file /home/priyadarshna/.zsh_history
(priyadarshna@kali) ~
$ gcc main.c geometry.c physics.c math_utils.c -o solid_projects
(priyadarshna@kali) ~
$ ./solid_projects

===== SOLID AND PHYSICS MENU =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: 8
Enter fluid density: 1000
Select solid (1-Sphere, 2-Cylinder, 3-Cone, 4-Cuboid): 1
Enter radius: 1
Buoyant Force = 41850.2400 N

===== SOLID AND PHYSICS MENU =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: 9
Enter solid density: 800
Enter fluid density: 1000
The solid floats.

===== SOLID AND PHYSICS MENU =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: 10
Exiting program...
(priyadarshna@kali) ~
$
```

*Test Case- 11,12,13:

```
Kali Linux
Session Actions Edit View Help
zsh: corrupt history file /home/priyadarshna/.zsh_history
(priyadarshna@kali) ~
$ gcc main.c geometry.c physics.c math_utils.c -o solid_projects
(priyadarshna@kali) ~
$ ./solid_projects

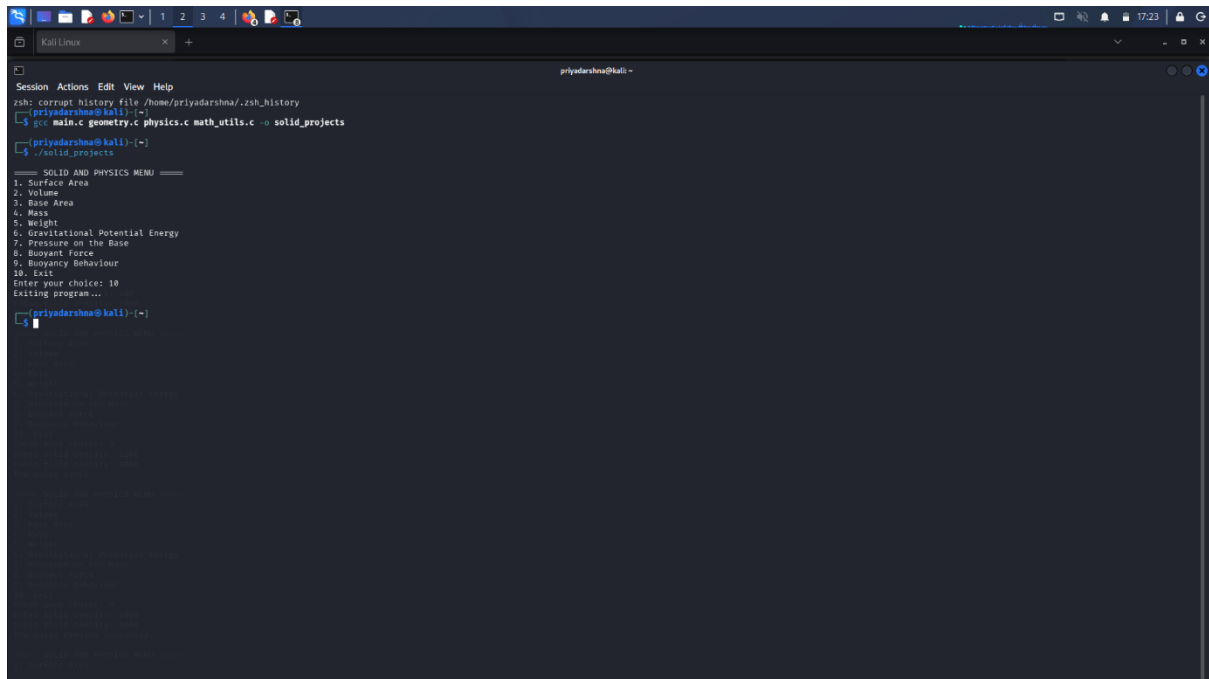
===== SOLID AND PHYSICS MENU =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: 9
Enter solid density: 500
Enter fluid density: 1000
The solid floats.

===== SOLID AND PHYSICS MENU =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: 9
Enter solid density: 1200
Enter fluid density: 1000
The solid sinks.

===== SOLID AND PHYSICS MENU =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: 9
Enter solid density: 1000
Enter fluid density: 1000
The solid remains suspended.

===== SOLID AND PHYSICS MENU =====
1. Surface Area
```

*Test Case 14:

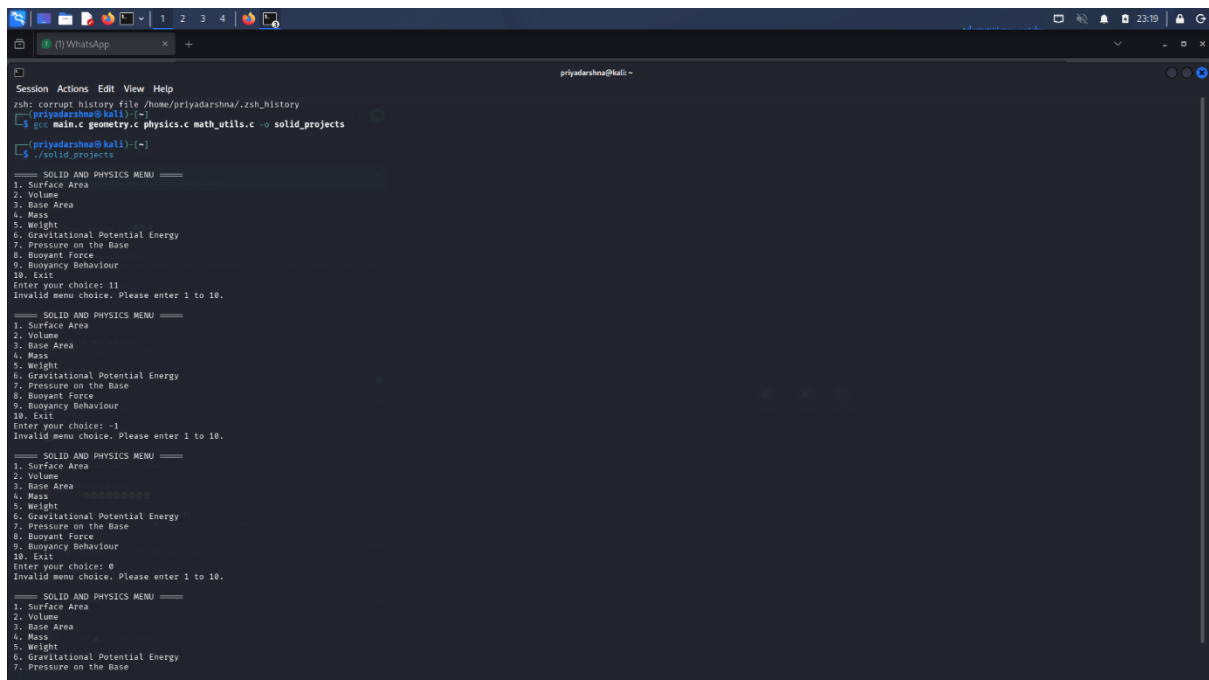


```
Session Actions Edit View Help
zsh: corrupt history file /home/priyadarshna/.zsh_history
(priyadarshna@kali) ~
$ gcc main.c geometry.c physics.c math_utils.c -o solid_projects
(priyadarshna@kali) ~
$ ./solid_projects

===== SOLID AND PHYSICS MENU =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: 10
Exiting program...

(priyadarshna@kali) ~
$
```

*Test Case -15,16,17:



```
Session Actions Edit View Help
zsh: corrupt history file /home/priyadarshna/.zsh_history
(priyadarshna@kali) ~
$ gcc main.c geometry.c physics.c math_utils.c -o solid_projects
(priyadarshna@kali) ~
$ ./solid_projects

===== SOLID AND PHYSICS MENU =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: 11
Invalid menu choice. Please enter 1 to 10.

===== SOLID AND PHYSICS MENU =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: -1
Invalid menu choice. Please enter 1 to 10.

===== SOLID AND PHYSICS MENU =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: 0
Invalid menu choice. Please enter 1 to 10.

===== SOLID AND PHYSICS MENU =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
```

*Test Case -18,19,20:

```
Kali Linux
priyadarshna@kali: ~
Session Actions Edit View Help
zsh: corrupt history file /home/priyadarshna/.zsh_history
priyadarshna@kali:~$ gcc main.c geometry.c physics.c math_utils.c -o solid_projects
priyadarshna@kali:~$ ./solid_projects

===== SOLID AND PHYSICS MENU =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: 28
Invalid menu choice. Please enter 1 to 10.

===== SOLID AND PHYSICS MENU =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: 9
Enter solid density: 800
Enter fluid density: 1000
The solid floats.

===== SOLID AND PHYSICS MENU =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: 9
Enter solid density: 1500
Enter fluid density: 1000
The solid sinks.

===== SOLID AND PHYSICS MENU =====
1. Surface Area
2. Volume
3. Base Area
```